IMPLEMENTATION HANDBOOK

MatchRead — Draw Automation, Reconciliation & Missing-Match Repair

Scope: close the two remaining manual gaps in the tournament pipeline — automatic discovery and publication of official draws, and repair of completed-match truth — so the existing lock, settlement and projection jobs run a tournament end to end without a human. Nothing else.

DATE	AUDIENCE	SITTING	BLOCKS	STATUS
23 Aug 2026	Launch engineer	One focused day	Winston-Salem · US Open	Ready to implement

> **Verification legend, stated once.** `main` is not reachable from this environment. Every path is marked **held** (file read directly), **`VERIFY`** (named across the corpus, source unseen), or **new**. Migrations `0006`, `0012`, `0013`, `0022`, `0025` and `0028`–`0030` are cited everywhere and present nowhere in this snapshot — confirm every column, enum label and function name against them before editing. **Where the repository contradicts a source document, the repository wins** and the correction is noted below.
>
> **Evidence base:** the Draw Discovery guide and the Reconciliation brief, re-verified line by line against `0001_init.sql` and `0004_security_hardening.sql`, plus `SETTLEMENT-SCHEDULING.md`, `ENGINEERING-HANDOFF.md`, the Product Constitution, the Tennis API integration brief, and the M3.5.2 / M3.5.3 / M3.6 provider milestones.

Corrections against the two source documents

#	SOURCE CLAIM	WHAT THE HELD FILES ACTUALLY SAY
1	Reconciliation §1: "*`match_status` (L18) has no `default`*"	A **type** cannot carry a default; `matches.status` (L100) is `not null default 'scheduled'`. The load-bearing half is confirmed: the enum has **no `suspended`** and no `default` value
2	Both docs imply triggers may gate a match write	**No trigger exists on `matches`.** Every trigger in `0001` (L261, L284) and `0004` (L108, L200, L260) targets `predictions`, `profiles` or `auth.users`. The only gates on a reconciliation write are the L112 check constraint and RLS — and `matches` has RLS enabled (L302) with a **select-only** policy (L316), so the pass must hold service-role
3	Reconciliation §5: walkover needs new "advances but does not grade" logic	Already shipped. `apply_grades` (`0004`) has a `voided` branch — `if v_voided then update matches set settled_at = now(); return 0;` — so `voided = true` **plus** a set `winner_side` already produces advance-without-grading. Write no new grading logic
4	Draw guide §2: `draw_seats` vs `draw_entries` / `draw_slots` unresolved	Still unresolved from held files. Evidence favours **`draw_seats`** (`SCREEN-INVENTORY`, `INTERACTION-SPEC`, `ENGINEER-MAPPING` and the Reconciliation brief); `PHASE-3-PROPOSAL` §1 is the lone dissenter. **`VERIFY`** against `0006` — do not carry both names forward

Confirmed exactly as the sources claim: `0001` L112 `check (status <> 'final' or winner_side is not null)`; L18 enum `('scheduled','live','final','walkover','retired','cancelled')`; L42 `players unique (external_ref, tour)` against L49/L94 **globally** unique `external_ref`; `0004` L494 `if v_settled is not null then return 0` — a latch, not a comparison.

1. Current failure and target state

The governing principle

> **Draw import owns bracket structure. Reconciliation owns completed-match truth. Settlement owns advancement.**

Three responsibilities, three writers, no overlap. Every design decision below follows from it, and the two most common ways to get this wrong — reconstructing a bracket from fixtures, and propagating winners into seats by hand — are each ruled out by one clause of it.

Draw-side failure — before play

Calendar ingestion is already automatic: Winston-Salem, Monterrey, Cleveland and both US Open events exist as `tournaments` rows with correct refs and dates. Draw creation is not. `/tournaments/t-atp-21348` renders *Draw pending* correctly, because no `draws` row for it has reached `published`.

Four `pg_cron` jobs are specified in `SETTLEMENT-SCHEDULING.md` §3 and every one presupposes a draw: `lock-draws` selects `where status = 'published' and now() >= locks_at`; `reconcile-results` posts results; `settle-tournament` grades an existing draw; `refresh-projections` refreshes a read model. **Nothing moves a tournament from no-draw to `published`.** The Product Constitution states the consequence plainly: *"Draws are inserted by hand."*

Result-side failure — after play

Advancement is a derivation, not a stored edge: one resolver runs over the seat table from the set of match results, so a single missing winner blanks every seat downstream of it. *"Not played"* is the correct rendering of that state — the bracket layer has no defect. Cincinnati produced **two distinct upstream shapes**:

Shape A — match exists locally, winner never resolved (ATP: Fonseca–O'Connell). `0001` L112 is the only outcome constraint and it requires a winner **only for `final`**. A row with `status='retired'` and `winner_side = null` is legal — nothing raises, nothing advances, and the `matches_settling` index re-selects it forever.

Shape B — completed match missing locally entirely (WTA: Wang–Svitolina). Confirmed against real captured bytes in M3.6: `/fixtures/tournament/{seasonId}` returns `{"data": [], "hasNextPage": false}` for a completed event. **The fixtures endpoint is a queue, not a register** — a match leaves it the moment it is played.

Two consequences define the fix: iterating local rows cannot repair Shape B (there is no row to iterate), and iterating upcoming-fixtures cannot repair it either (the match is gone from that endpoint permanently, so the comparison sees nothing on both sides and correctly concludes nothing is missing). The authoritative repair source is therefore the completed-results archive, and only that.

Target lifecycle

```
calendar ingest        tournament row exists                    [already automatic]
  → discover-draws     Draw API polled, official structure found  [PART A — new]
  → import + validate  one transaction, fail-closed
  → publish            draws.status = 'published'
  → lock-draws         published → locked at first ball          [unchanged]
  → play
  → reconcile          completed-results sweep repairs matches   [PART B — new/finish]
  → settleAll          existing resolver fills every seat        [unchanged]
  → projections        leaderboard refresh                       [unchanged]
```

2. Existing code to reuse

Reuse — confirmed across the corpus:

PATH	STATE	WHAT YOU TAKE FROM IT
`packages/core/src/canon/tennis.ts`	VERIFY	Draw-shape model (4–128, byes, rounds, opening round, seed count) and `matches = size − 1`, M3.6's completeness check. Also already holds *"a retirement grades and a walkover voids"* — make it callable as `outcomeDisposition(resultType)`
`packages/provider-rapidapi/src/client.ts`	VERIFY	`getTournamentResults(tour, seasonId)`. **Add only if absent** — M3.6 generated a 20-field spec from 27 real payloads at `/tournament/results/{id}`, so a client method very likely exists. Enforce lowercase tour in the path
`packages/provider-rapidapi/src/parse.ts`	VERIFY	The five aliases confirmed against real bytes: `result_type` (**not** `status`), `player1Id`, `match_winner` (a player id, not a side), `countryAcr`, `result`
`packages/settlement`	VERIFY	`settleAll(...)`. Also **the pattern to copy** — logic extracted so it runs in Node and CI outside Supabase
`apps/mission-control/provider.mjs`	VERIFY	The single provider descriptor. Host, base path, `{type}` tour substitution

PATH	STATE	WHAT YOU TAKE FROM IT
`scripts/lib/strict-provider.mjs`	`VERIFY`	The hostile fake — 404s unknown routes, reproduces `pageSize + 1` and empty-fixtures-on-completed-event
`scripts/geneva-verify.mjs`	`VERIFY`	The shape to copy for verification against real bytes
`apps/web/server/repositories/founder.ts`	`VERIFY`	Dashboard thresholds live in this one file by existing rule
`.github/workflows/`	`VERIFY`	The only scheduler documented as *working* that resolves `workspace:*`
`0001_init.sql`	**held**	`tournaments(external_ref unique, tour, tier, starts_on, ends_on, draw_size, season_id)`; `matches(external_ref unique, status, winner_side, model_probability_a, voided, settled_at)`; `players unique (external_ref, tour)`
`0004_security_hardening.sql`	**held**	`apply_grades` — the latch at L494 and the `voided` branch that already implements advance-without-grade

Does not exist — do not go looking: any draw importer or `scripts/import-*`; `apps/listener`, `Dockerfile`, `railway.json`; error tracking or analytics in `apps/web`; a reconciliation audit table in anything held. **Enums confirmed absent:** `match_status` has no `default` and no `suspended`.

3. Part A — automatic draw discovery and import

3.1 Pin the Draw API — blocking, ~30 minutes

The principle *do not reconstruct the bracket from fixtures when the authoritative Draw API is available* is correct and is **not yet expressed anywhere in the repository**. The integration brief's Phase 2 is fixtures-based and predates the finding; its own §3C lists draw order, slot numbers, bye positions and qualifier placeholders as **unverified**, and its Question 01 to provider support is literally *"which endpoint returns the official pre-tournament main draw in bracket order?"* Treat §3C/Phase 2 as superseded; M3.6 records the same gap — *"no endpoint has ever supplied a draw structure."*

Before writing a parser, confirm and record: exact route, method, required provider ids, singles/main-draw filter, **one real ATP payload and one real WTA payload**.

1. Add the draw route to the descriptor in `apps/mission-control/provider.mjs`, using the same `{type}` substitution as the other routes. M3.5.2 is the cautionary tale: `planFor()` ran `step.path.replace('{type}', tour)` against paths containing no `{type}` and the no-op went unnoticed for the life of the project. **Assert the substituted URL, not the template.**
2. Add the matching anchored pattern to `scripts/lib/strict-provider.mjs`. That fake 404s anything unrecognised by design; until the draw route is in its table every test fails for the wrong reason.
3. Capture both payloads with the existing `capture.mjs` (`--tour wta` also discharges the never-exercised WTA path) and generate their spec into `docs/provider-knowledge/`.

Do not start 3.2 until real payloads are on disk. The last time this project wrote a parser from documentation, it scored **0 of 27**.

3.2 One shared importer — `packages/draw-import/` (new)

Mirror `packages/settlement`: pure logic, own client factory, executable in Node and CI.

```
// packages/draw-import/src/index.ts
export async function importOfficialDraw(args: {
  tournamentRef: string;              // 't-atp-21348'
  source: 'automatic' | 'manual';
  client: SupabaseClient;
  now: Date;                          // never read a clock — settleAll's rule
}): Promise<
  | { outcome: 'published';       drawId: string; slots: number }
  | { outcome: 'not_yet' }        // provider has no draw. Safe no-op, the common case
  | { outcome: 'already_present' }
  | { outcome: 'rejected';        reason: string; payloadRef: string }
>;
```

Four outcomes, no exceptions for normal control flow. **Provider identity is nearly free:** `tournaments.external_ref` is `text not null unique` (`0001` L49) and production refs read `t-atp-21348` / `t-wta-16741` — the tour and provider season id are already in the string. Parse the ref; fall back to the calendar endpoint only on a malformed one, and log a mapping failure rather than guessing.

3.3 Discovery worker

```
// packages/draw-import/src/discover.ts
export async function discoverDraws({ client, now }: Ctx) {
  for (const t of await findDrawDiscoveryCandidates(client, now)) {
    const r = await importOfficialDraw({
      tournamentRef: t.external_ref, source: 'automatic', client, now,
    });
    const ref = t.external_ref;
    if (r.outcome === 'rejected')  log.error({ ref, reason: r.reason, payload: r.payloadRef });
    if (r.outcome === 'published') log.info({ ref, drawId: r.drawId, slots: r.slots });
    // 'not_yet' and 'already_present' are silent. A quiet run is the normal run.
  }
}
```

An alarm that fires on the normal case stops being read. This project has shipped that defect four times by its own count — `late_arrival`, duplicate-as-quarantine, the coupling scanner's substring matches, and the fixtures warning. Do not make it five.

Eligibility. Held columns and enums only — `tour('ATP','WTA')`, `tournament_tier('grand_slam','tour_finals','masters_1000','tour_500','tour_250','other')`:

```
select t.id, t.external_ref, t.tour, t.tier, t.draw_size, t.starts_on
from tournaments t
where t.tour in ('ATP','WTA')
  and t.tier not in ('tour_finals','other')          -- round-robin: canon says isKnockout:false
  and t.starts_on between current_date - 2 and current_date + 21
  and not exists (
    select 1 from draws d                            -- VERIFY status labels against 0006
     where d.tournament_id = t.id
       and d.status in ('published','locked','complete'))
order by t.starts_on;
```

If `0006` already exposes a canonical eligibility view or a `bracket_eligible` field, **use it and delete this query** — tier policy must not live in two places. Two notes: qualifying and doubles cannot be excluded here (the calendar creates one row per `(tour, season id)`, so there is no doubles row to filter) — exclusion happens at the payload boundary via a singles-only request plus `looksLikeDoublesPair`, which caught that 16 of 56 captured "players" were pairs. And **replay draws must be excluded**: mirror whatever predicate `settleAll` uses (`SETTLEMENT-SCHEDULING.md` §"How active draws are selected"); migration `0022` exists so a rehearsal cannot contaminate production.

3.4 Validation and publication

One transaction, advisory lock, fail closed.

```
begin;
  select pg_advisory_xact_lock(hashtext(:tournament_ref));
  archive raw payload (hash-chained, before parsing)
  insert draw (pre-publication status VERIFY) + seats
  validate  ── any failure ──▶ rollback: zero rows, tournament stays Draw pending
  set status = 'published', structure_provenance = <provider-authoritative>
commit;
```

Validate exactly these seven, and no more:

1. **Identity** — payload season id equals the one parsed from `external_ref`; tour matches.
2. **Main singles** — not qualifying, not doubles; every name passes `looksLikeDoublesPair`.
3. **No duplicate draw positions** — a set check over slots.
4. **No conflicting authoritative draw** — the `not exists` from 3.3, re-asserted inside the transaction under the lock.
5. **Internally parseable** — the canon resolves a shape: `matches = size − 1`, byes placed, rounds derivable. Round ids are **not contiguous** (Geneva: 4, 5, 9, 10, 12) and the provider's table holds **two quarter-final ids** — map through the canon, never by arithmetic.
6. **Slot count reconciles** to `tournaments.draw_size` (`not null` in `0001` L58). Note the live conflict: the calendar carries `draw_size` *"when populated"* while M3.6 found no draw-size field anywhere. If they disagree at runtime, **reject** — that disagreement is exactly the defect class this gate exists to catch.
7. **Provenance** — provider-authoritative, never `reconstructed`. `ENGINEERING-HANDOFF.md` is explicit that a reconstructed draw must never be presented as authoritative. `VERIFY` the enum labels in `0025`.

On failure: roll back, log `{ ref, reason, payloadRef }`, keep the raw payload. **Use the existing lifecycle** — `published` is what `lock-draws` already looks for. Do not invent a status.

Seeds. M3.6: seed *count* is a tour rule, byes identify seeds 1–N *as a set*, and which one is seed 1 is not recoverable by arithmetic. Take seeds if the Draw API supplies them. If not, MatchRead may say *"these four were seeded 1 to 4"* and must not say *"this one was seed 1"* — the difference between a limitation and a fabrication.

3.5 Manual recovery

```
pnpm draw:import --tournament=t-atp-21348      # source: 'manual'
```

Same `importOfficialDraw`, same transaction, same validation. `source` is recorded and changes nothing about behaviour. **One implementation, two callers** — this is the repair tool when the workflow is red at 3am.

4. Part B — reconciliation and missing-match repair

4.1 Authoritative source

`GET /tennis/v2/{tour}/tournament/results/{seasonId}` (brief §3E), singles only — the completed-results archive, where `player1` is **always** the winner and `result_type` carries `completed | retired | walkover | default`. It is the only endpoint that still holds a played match.

4.2 Behaviour

```
for each active supported tournament
  → GET /{tour}/tournament/results/{seasonId}          (singles only)
  → normalize (result_type, match_winner, player1Id/player2Id)
  → match locally on (tour, providerMatchId)           ← identity, never names
      ├─ no local row          → create from official result (if safe)
      ├─ local winner null     → fill winner + status
      ├─ local winner differs  → record conflict, write nothing
      └─ agrees                → no-op
  → settleAll({ drawIds: [t.drawId] })                 ← EXISTING engine
  → one run row + one audit row per change
```

4.3 Code changes

#	PATH	STATE	CHANGE
1	`packages/provider-rapidapi/src/client.ts`	`VERIFY`	`getTournamentResults(tour, seasonId)` — reuse if present
2	`packages/provider-rapidapi/src/parse.ts`	`VERIFY`	`normalizeResult(raw)`. Unknown `result_type` → throw `UnknownOutcome`. **Never read `result`** — free-form, 24 distinct values in 27 payloads
3	`packages/core/src/canon/tennis.ts`	`VERIFY`	`outcomeDisposition(resultType)` — the §4.4 table as a function
4	`packages/settlement/src/reconcile.ts`	**new**	`reconcileTournament()`, `reconcileActive()`
5	`packages/settlement/src/repo.ts`	`VERIFY`	`findMatchByProviderIdentity({tour, providerMatchId})` — **requires migration 3**
6	`packages/settlement/src/index.ts`	`VERIFY`	export `reconcileTournament`, `reconcileActive`
7	`apps/web/server/repositories/founder.ts`	`VERIFY`	Reconciliation tiles + thresholds

Migrations — `0031_reconciliation.sql` (head is `0030`). Each is justified by a held file; if `main` already ships one, drop it.

#	CHANGE	PROOF
1	`alter type match_status add value 'default', 'suspended'`	`0001` L18 has neither; provider emits `default`, brief §6 documents `SUSPENDED`. Cast raises `22P02` today. **Ship in its own step** — `ADD VALUE` cannot be used in the same transaction that queries the new value
2	Replace L112 with `status not in ('final','retired','default','walkover') or winner_side is not null`	L112 covers `final` only. This is the exact state ATP Cincinnati is in and it must become unstorable. `cancelled` / `suspended` excluded — neither has a winner
3	`matches` / `tournaments`: drop global unique on `external_ref`, add `unique (external_ref, tour)`	L94/L49 are globally unique; L42 already scopes `players` by tour. Provider id spaces are per-tour, and two 128-draws run in parallel at the US Open. **Run the collision report before dropping**

#	CHANGE	PROOF
4	`reconciliation_runs`, `reconciliation_repairs`, `matches.reconciled_at`	No audit table exists in anything held. Service-role RLS, founder-readable. **If an ingest audit table exists on `main`, use it instead**

pgTAP `31_reconciliation.test.sql` ships in the same commit.

4.4 Core implementation

```
// packages/settlement/src/reconcile.ts
export async function reconcileTournament(t: ActiveTournament, run: RunContext) {
  const raw     = await provider.getTournamentResults(t.tour, t.providerSeasonId);
  const results = raw.filter(isSingles).map(normalizeResult);      // throws UnknownOutcome

  for (const r of results) {
    const local = await repo.findMatchByProviderIdentity({
      tour: t.tour,                       // identity is (tour, providerMatchId).
      providerMatchId: r.providerMatchId,  // never a name, never a position
    });

    const disposition = outcomeDisposition(r.resultType);          // canon, §4.5
    if (disposition.kind === 'unknown') { await run.quarantine(r); continue; }
    if (!local) { await createMissingMatchFromOfficialResult(r, t, disposition, run); continue; }

    const side = await resolveWinnerSide(local, r);                // provider id → 'a' | 'b'
    if (side === 'conflict')            { await run.conflict(local, r, 'roster_mismatch'); continue; }
    if (!local.winnerSide && side)      await repairOfficialResult(local, r, disposition, side, run);
    else if (local.winnerSide && side && local.winnerSide !== side)
                                        await run.conflict(local, r, 'winner_changed');
    // agrees → no-op.  Fill a hole: automatic. Change an answer: never.
  }

  await settleAll({ drawIds: [t.drawId] });   // EXISTING engine. Seats fill here, not above.
  await run.finish();
}
```

`createMissingMatchFromOfficialResult` fails closed. `matches` requires `player_a_id`/`player_b_id` (FK), `model_probability_a` (`not null`, 0–1), `starts_at`, `round`. If a player is unknown, fetch and insert the **real** player — never a placeholder; if it cannot be fetched, record `player_unresolved` and write nothing. For `model_probability_a` insert `0.5000` **with `voided = true`**: the bracket advances (the resolver needs only a winner) and no rating moves on a match nobody could have predicted. **Never write `0.5` without the void flag.**

The blocker to check first. `apply_grades` (`0004` L494) guards with `if v_settled is not null then return 0` — a **latch, not a comparison**, and its own comment explains why (the rating ledger is append-only, so a duplicate is unrecoverable). If `apply_bracket_state` (`0012`) shares that shape, a repaired winner on an already-`settled_at` match is a silent no-op and this entire pass reports success while changing nothing. **Read `0012` before writing anything**; if it latches, add the third branch — *settled + outcome differs → raise, record conflict.*

4.5 Exceptional outcomes

Rule under the table: **did tennis get played?** If yes there is a winner and the pick grades. If no there may still be a winner and the pick voids. `winner_side` drives advancement; `voided` drives grading. They are separate questions.

MATCHSTAT	ADVANCES	GRADES	`MATCHES.STATUS` / `VOIDED`	NOTE
`completed`	**Yes**	**Yes**	`final` / false	—
`retired`	**Yes**	**Yes**	`retired` / false	Canon: a retirement grades
`default`	**Yes**	**Yes**	`default` / false	Needs migration 1. Extends the canon by one case — get a founder line in `PRODUCT_DECISIONS.md`
`walkover`	**Yes**	No	`walkover` / **true**	A walkover **still advances**. Treating it as unresolved is how one absence blanks a quarter of the draw. `apply_grades`' existing `voided` branch already produces this — write no new logic

MATCHSTAT	ADVANCES	GRADES	MATCHES.STATUS / VOIDED	NOTE
`cancelled`	No	No	`cancelled` / true	Renders “Cancelled”, **not** “Not played”
`suspended`	No	No	`suspended` / false	Not terminal. Never settle, never latch `settled_at`
anything else	No	No	**no write**	Quarantine, raise, alert. Never default to `final`, never guess

Winner identity comes from `match_winner` (a provider player id), corroborated against `playerId`. **Disagreement is a conflict, not a tiebreak.** Never parse the score string.

4.6 Settlement — the rule that must not bend

Reconciliation never writes a bracket seat. It writes `matches` only. Once `winner_side` is correct, `settleAll` re-runs the existing resolver and every downstream seat fills. There is no propagation step to build, and building one would create a second source of bracket truth — the precise failure §1 exists to prevent.

5. Shared safety, idempotency and audit

These rules govern both parts. They are stated once here and not repeated.

Identity. Provider numeric ids, scoped by tour, always. Never names: 39 shared surnames among 900 ranked players including four sets of brothers, zero diacritics (the provider strips them), and country alone produced six spurious ambiguities in a 28-player draw. Never positions, never score strings.

Fail closed. Ambiguity means reject, quarantine or conflict — never guess. A partially imported bracket must never become user-visible; a match with an unresolvable player must never be written.

Idempotency. Both passes must be replay-safe by construction, not by convention.

	DRAW DISCOVERY	RECONCILIATION
Before data exists	50 runs → **0 draws**, 0 rows, 0 errors	Nothing to repair → 0 writes
Data appears	**1 draw**, published, provenance correct	Only missing state created or filled
Repeat ×3	**still 1 draw**, 1 seat set	**0 further writes**
Concurrent runs	**1 draw** (advisory lock)	No overlap (advisory lock per `seasonId`)

Enforcement is structural: `pg_advisory_xact_lock` per tournament or `seasonId`; a unique index on `draws(tournament_id)` (`VERIFY 0006`, add in `0031` if absent); a uniquely-keyed content hash on each `reconciliation_repairs` row so a replay cannot double-write; and workflow-level `concurrency` groups.

Raw payload preservation. Archive provider bytes **before** parsing, hash-chained, per existing discipline. This has already paid for itself once: Geneva’s parser scored 0 of 27 and only re-read cleanly after correction because the raw bytes were kept. Capture Cincinnati ATP + WTA into `fixtures/cincinnati-2026/` before writing the reconciliation parser.

Audit. Every meaningful write traceable to tournament, provider record, run id, before state, after state, source and timestamp. Reuse an existing ingest audit table if `main` has one; otherwise migration 4 above creates the pair. Draw imports record `source: 'automatic' | 'manual'`.

6. Scheduling and operational flow

The decision rule, stated once, because it governs both parts. `SETTLEMENT-SCHEDULING.md` §STOP documents that the settlement Edge Functions import unpublished `npm:` packages — `@matchread/core` depends on `@matchread/i18n` through `workspace:*`, which npm cannot resolve — so they **deploy green and 500 on first invocation**. Both `discoverDraws` and `reconcileActive` depend on the canon and the settlement package, so both inherit that failure verbatim if hosted as Edge Functions.

> **So:** if §STOP is already closed on `main` (option A published packages, or B vendored bundles), host reconciliation on the existing `reconcile-results` cron → `ingest-events` with a `{"mode":"reconcile"}` branch, exactly as the source brief specifies, and add one cron target for discovery. **If §STOP is still open**, run both as monorepo CLI entry points from GitHub Actions — the only scheduler documented as working that resolves `workspace:*` and holds secrets. **Check this first; it changes two files, not the design.** When the Railway container lands (ADR-0018 Tier 1), both invocations move into it and the workflows are deleted.

Two commands, two schedules, two concurrency groups — **not one all-purpose worker.** Separate cadences and separate failure domains.

STAGE	CADENCE	ACTION	STATUS
Calendar ingest	existing	tournament row exists	unchanged

STAGE	CADENCE	ACTION	STATUS
Draw discovery	`*/5`	poll Draw API for eligible tournaments	**new target**
Draw import	on discovery	validate → publish, one transaction	**new**
`lock-draws`	`*/1`	`published` → `locked`	unchanged
Reconciliation — on court	`*/5`	results + fixtures	**extends `reconcile-results`**
Reconciliation — finished < 24 h	`*/30`	results only — where late corrections land	**new state**
Reconciliation — upcoming, draw published	`*/60`	fixtures only	**new state**
`settle-tournament`	`*/15`	`settleAll` derives advancement	unchanged
`refresh-projections`	`*/1`	leaderboard refresh	unchanged

Request budget. ~2 requests per tournament per pass at `pageSize=500` (a 128-draw is 127 matches — one page). Busy fortnight, 6 events on court: ≈ **158 req/hr**, largest burst 12 against a documented **100 req/min** throttle. Per-minute headroom is fine; the monthly plan cap is not verified — **confirm ≈114k requests/month with the founder.** Limit globally to 60 rpm and stagger by `hash(seasonId) % 60`.

Verify the scheduler before trusting it. Every `pg_cron` call in the repository is documented as a comment, in three places. Run `select jobid, jobname, schedule, active from cron.job` and `select * from cron.job_run_details order by start_time desc limit 20`. If cron fires but the run table is empty, the function is 500ing — see §STOP above.

7. Today's implementation sequence

1. **Verify Git assumptions (60 min).** Draw table and status enum, seat table name (`draw_seats` vs `draw_slots`), provenance enum in `0025`, whether `0012`'s `apply_bracket_state` latches like `apply_grades`, `cron.job` state, provider client path, existing audit tables. **Nothing below is safe until this is done.**
2. **Pin the Draw API (30 min).** ATP + WTA payloads captured; descriptor and strict fake updated; spec generated.
3. **Build `importOfficialDraw` (2 h).** Validation, transaction, advisory lock, four outcomes.
4. **Prove manual import.** `t-atp-21348` and `t-wta-16741`. Both render on the production tournament page.
5. **Prove draw idempotency.** Re-run ×3 and concurrently — draw count unchanged.
6. **Wire unattended discovery.** Schedule per §6; then prove a *future* tournament is found with nobody watching.
7. **Implement reconciliation (2 h).** Migration `0031` (`ADD VALUE` in its own step), `reconcile.ts`, `normalizeResult`, `outcomeDisposition`.
8. **Prove settlement reuses repaired truth.** One repaired R32 winner → downstream "Not played" seats resolve.
9. **Prove reconciliation is a no-op on replay.** Three further scheduled runs, zero writes.
10. **Final production check.** Both schedulers firing, audit rows visible, both manual commands available, `lock-draws` / `settle-tournament` / `refresh-projections` untouched.

Steps 4–6 gate Winston-Salem; steps 7–9 gate Cincinnati. If the day runs short, **stop after step 6** — a published draw with manual reconciliation is a working product; automatic reconciliation over an absent draw is not.

8. Tests and definition of done

Fixtures are real bytes or they are worthless. Synthetic fixtures cannot reproduce either Cincinnati defect, and a permissive fake certifies bugs and issues a green tick with them — the M3.5.2 finding that produced `strict-provider.mjs`.

#	TEST	ASSERTION
D1	Draw unavailable	50 runs → 0 rows of any kind, 0 errors logged
D2	Draw appears	1 draw published, provenance provider-authoritative
D3	Repeat ×3 / concurrent	Still 1 draw, 1 seat set, no duplicates
D4	Malformed positions / slot-count mismatch	Rejected; zero rows; reason + `payloadRef` logged
D5	Wrong event type	Qualifying, doubles, unsupported tour, completed event → not imported, no error noise
R1	Missing local match discovered	Created from results; both players resolved by provider id; `action='created'`; downstream seat resolves after settle. **Must fail before the fix**
R2	Null winner repaired	Winner filled; `action='winner_filled'`; settles; no other row touched

#	TEST	ASSERTION
R3	Walkover advances	`status='walkover'`, `winner_side` **set**, `voided=true`, **next-round seat occupied**, no prediction graded. *The seat assertion is the test*
R4	Retirement advances	`status='retired'`, winner from `match_winner`, advances, **grades**, not voided
R5	Unknown status quarantines	`result_type:"abandoned"` → raises, no winner written, alert
R6	Conflict does not overwrite	Different provider winner on a settled match → conflict row, S1 alert, `matches` unchanged. **Mutation check:** delete the conflict branch and confirm R6 fails
R7	Settlement is reused	Reconciliation writes to no seat table (assert **and** grep); `settleAll` called once per affected draw
R8	Replay ×3 is a no-op	Runs 2–3 write zero rows — assert `max(matches.updated_at)` unchanged, not merely "no error"
P1	pgTAP	ATP ref `N` and WTA ref `N` both insert (migration 3); `insert … status='retired', winner_side=null` **rejected** (migration 2)

P1's second assertion is the regression test for the whole incident. If it already passes on `main`, the constraint is stronger than `0001` and Shape A has a different cause — worth ten minutes before writing migration 2.

Production proof. Deploy, touch no row by hand, let the schedulers do it. Draw side: Winston-Salem and Monterrey publish and render, then one future tournament is discovered unattended. Result side: the ATP branch resolves (`winner_side` non-null, audit row names the action), the WTA branch resolves (`action='created'`, `external_ref` matches the provider id), downstream seats populate, and three further runs change nothing. **Falsifiers:** branches heal with an empty audit table → something else healed them. Healing only after manual invocation → the scheduler is unproven. **No manual SQL update counts as proof.**

If production Cincinnati cannot be used safely, point reconciliation at a completed tournament MatchRead has never ingested — Geneva 2025 from the existing capture, or Toronto/Montreal — on a **replay draw**. Migration `0022`'s isolation exists for exactly this. Do **not** delete a production row to simulate a gap.

Definition of done

- [ ] Supported ATP/WTA tournaments are checked automatically for official main draws
- [ ] Draw API data imported using provider-authoritative structure — never reconstructed from fixtures
- [ ] A draw is validated before it becomes `published`; partial brackets never render
- [ ] Repeated or concurrent discovery cannot create duplicates
- [ ] Winston-Salem `t-atp-21348` and Monterrey `t-wta-16741` import and render correctly
- [ ] One future tournament proves unattended discovery
- [ ] Manual `pnpm draw:import --tournament=<ref>` remains, calling the same `importOfficialDraw`
- [ ] Reconciliation finds both missing matches (Shape B) and missing winners (Shape A)
- [ ] Conflicting provider results record instead of overwriting; unknown outcomes quarantine
- [ ] Exceptional outcomes behave per §4.5; migration `0031` applied
- [ ] **Reconciliation writes no bracket seat**; `settleAll` derives every advancement
- [ ] Repeated reconciliation runs are no-ops, asserted on `max(matches.updated_at)`
- [ ] Both schedulers proven firing in production, with audit rows
- [ ] `lock-draws`, `settle-tournament` and `refresh-projections` unchanged